Foundations of Automatic Verification
Assignment Report

Vipul Nayyar (2024CSZ8495)

CNF Encoding

The solution accepts an N X N array. Each field(i,j) in the array is encoded so that a unique SAT literal can be associated with each field. The field encoding is done as below, where i is the row index in the 2D input array starting from 0, and y is the column index. An example of this encoding is present in Figure 1.

$$\text{encoded_address}(i , j) = 1 + (i * \text{array_size}) + j$$

Let us assume that each field is represented by a literal A(i,j) that is obtained from the above encoded address.

The following clauses are used to build CNF encoding using PySAT[1]:

1) The start field(0,0) and end/goal field(N-1, N-1) have to be True as they will be part of the path to be found.
   - A(0,0)
   - A(N-1 , N-1)

2) Each field with an obstacle must not be true, as it cannot be part of the path.
   - ¬A(i,j) where field in input array at i , j is an obstacle('X')

3) If a field is chosen for the path, at least one of its right or bottom neighbors must also be chosen unless it is the bottom right goal field (N-1, N-1).
   - If a field chosen(i,j) is in the last column, A(i+1 , j) must be True.

     $A(i , j) \rightarrow A(i+1 , j) \equiv \neg A(i,j) \vee A(i+1,j)$

   - If a field chosen (i,j) is in the last row, A(i,j+1) must be True.

     $A(i , j) \rightarrow A(i , j+1) \equiv \neg A(i,j) \vee A(i,j+1)$

   - If a field(i,j) is neither in the last row nor the last column, then one of A(i,j+1) or A(i+1,j) must be True.

     $A(i , j) \rightarrow (A(i,j+1) \vee A(i+1,j)) \equiv \neg A(i,j) \vee A(i , j+1) \vee A(i+1 , j)$

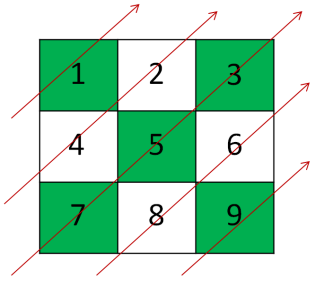


Figure 1: Diagonals of a Matrix and its associated Encoded address [2]

4) If we divide the array into diagonals as shown in Figure 1, we can assume that each move(right or down) will consist of one node in the previous diagonal and one node in the next diagonal. Therefore the path will consist of **exactly one node** from each diagonal.

   - For all n fields in every diagonal, **at least** one field in every diagonal should be True $(A(i_0 , j_0) \vee A(i_1 , j_1) \vee \ldots\ldots \vee A(i_n , j_n))$ for all k belonging to Diagonals $\{D_1,D_2 \ldots D_k\}$

   - For all n fields in every diagonal, **at most** one field in every diagonal should be True. For this, in every diagonal, for each possible pair of fields, both cannot be True

     $(\neg A(i_0 , j_0) \vee \neg A(i_1 , j_1)) \wedge (\neg A(i_0 , j_0) \vee \neg A(i_2 , j_2)) \wedge \ldots \wedge (\neg A(i_0 , j_0) \vee \neg A(i_n , j_n)) \wedge$
     $(\neg A(i_1 , j_1) \vee \neg A(i_2 , j_2)) \wedge (\neg A(i_1 , j_1) \vee \neg A(i_2 , j_3)) \wedge \ldots \wedge (\neg A(i_1 , j_1) \vee \neg A(i_n , j_n)) \wedge$
     $\ldots \wedge (\neg A(i_{n-1} , j_{n-1}) \vee \neg A(i_n , j_n)$ … for all k belonging to Diagonals $\{D_1,D_2 \ldots D_k\}$

Final CNF Encoding:

$A(0,0) \wedge A(N-1 , N-1)$
$\wedge$
$\neg A(i_1 , j_1) \wedge \neg A(i_2 , j_2) \wedge \ldots \wedge \neg A(i_n , j_n)$ for all obstacles
$\wedge$
$\neg A(i,j) \vee A(i , j+1) \vee A(i+1 , j)$ for all fields not in last row or column
$\wedge$
$\neg A(i,j) \vee A(i+1,j)$ for all fields in last column
$\wedge$
$\neg A(i,j) \vee A(i,j+1)$ for all fields in last row
$\wedge$
$(A(i_0 , j_0) \vee A(i_1 , j_1) \vee \ldots\ldots \vee A(i_n , j_n))$ for all k belonging to Diagonals $\{D_1,D_2 \ldots D_k\}$
$\wedge$
$(\neg A(i_0 , j_0) \vee \neg A(i_1 , j_1)) \wedge (\neg A(i_0 , j_0) \vee \neg A(i_2 , j_2)) \wedge \ldots \wedge (\neg A(i_0 , j_0) \vee \neg A(i_n , j_n))$
$\wedge$
$(\neg A(i_1 , j_1) \vee \neg A(i_2 , j_2)) \wedge (\neg A(i_1 , j_1) \vee \neg A(i_2 , j_3)) \wedge \ldots \wedge (\neg A(i_1 , j_1) \vee \neg A(i_n , j_n))$
$\wedge \ldots \wedge (\neg A(i_{n-1} , j_{n-1}) \vee \neg A(i_n , j_n)$ … for all k belonging to Diagonals $\{D_1,D_2 \ldots D_k\}$

Incremental SAT solution uses Cardinality Constraints provided by pysat.card.ITotalizer[3]

<u>SMT Encoding</u>

For SMT Encoding, each field of the input array is considered as a node in a Graph using Z3[4]. Each node has an edge connecting to its bottom and right neighbor in the input array. If the bottom or right neighbor is an obstacle or does not exist, it is stored as 'nil'. The SMT constraints try to build a path that connects the start node and the goal node, which is of length(2 * length of array - 1).

- A Z3 datatype called 'GraphNode' is created that stores each nodes row index, column index, right neighbor link and bottom neighbor link.

```
GraphNode = Datatype('GraphNode')
GraphNode.declare('data', ('row', IntSort()), ('col', IntSort()), ('rightnext',
GraphNode), ('bottomnext', GraphNode))
GraphNode.declare('nil')
```

- A 2D array called 'graph_nodes' is used to store all GraphNodes for all fields in the input array that store edges between nodes(right and bottom).
- A 1D array called 'graph_path' of length(2*N - 1) is used to build path from start to goal.

The following constraints are used to build the SMT Encoding:

1) If a field (i , j) in input array is an obstacle,

   graph_nodes[i][j] = nil

2) If a field (i , j) is not an obstacle,

   graph_nodes[i][j] = GraphNode.data(i , j , right_neighbor, bottom_neighbor))

3) The first element of graph_path array is start field.

   graph_path[0] = graph_nodes[0][0]

4) The last element of graph_path array is goal field.

   graph_path[2 * length of array - 2] = graph_nodes[length of array - 1][length of array - 1]

5) No element in graph_path is allowed to be nil.

   graph_path[i] != GraphNode.nil

6) For all elements in graph_path except for last element, the next element in the array must be a neighbor of the current element:

   Or(
   graph_path[i + 1] == next_right(graph_path[i]),
   graph_path[i + 1] == next_bottom(graph_path[i])
)

NuSMV Model

The NuSMV[5] model consists of two traffic lights(NSLight and EWLight) which control North-South and East-West traffic. Each cardinal direction(4) has a queue of cars associated with it, which allows cars to pass straight through. The following Modules and their associated variables are used to track states:

1) MODULE **'main'**: Manages two traffic lights using a token value. It ensures that at a time, only one light can have its token = 1, which enables a light to become green
   - NSLight, EWLight, NorthToSouthQueue, SouthToNorthQueue, EastToWestQueue, WestToEastQueue

2) MODULE **'light'**: Manages a traffic light using a tick-based counter which goes from 1 to 12 when its token is enabled by the main module.
   - token, counter, state

Each value of the counter is associated with the light's state changes from red → yellow → green → yellow → red. The light is red when counter is 1 or 12, the light is yellow when the counter is 2,3,10 or 11, and the light is green when the counter is in 4 - 9. When the counter reaches its last state(12), its token value is changed to 0, and the other light's token value is changed to 1 so that the other light can start its counter tick from 1 to 12.

3) MODULE **'queue'**: Four queues are created for traveling in each direction by the main module. Each queue has 5 cars. Each car can have a status of ACTIVATED or DEACTIVATED. Each car can have a state of WAITING, PASSING, or CLEARED.
   - Carstate[ ], carstatus[ ], startindex, reset

Once all cars have cleared from the queue, the queue is reset, and all five cars are again ready to pass through its associated traffic light.

CTL Properties

- **Fairness**:

```
AG(
      AF(NSLight.state = green & EWLight.state != green)
      &
      ((NSLight.state = green & EWLight.state != green) -> AF ( EWLight.state = green &
NSLight.state != green ) )
      &
      AF(EWLight.state = green & NSLight.state != green)
      &
      ((EWLight.state = green & NSLight.state != green) -> AF ( NSLight.state = green &
EWLight.state != green ) )
   )
```

- **Liveness**: The following CTL properties verify Liveness for 5 cars used in the queue repeatedly.

```
CTLSPEC
   AG (
      AF(carstate[0] = CLEARED & light.state = green)
      &
      AF(carstate[0] = WAITING -> AF(carstate[0] = CLEARED & light.state=green) )
   )

CTLSPEC
   AG (
      AF(carstate[1] = CLEARED & light.state = green)
      &
      AF(carstate[1] = WAITING -> AF(carstate[1] = CLEARED & light.state=green) )
   )

CTLSPEC
   AG (
      AF(carstate[2] = CLEARED & light.state = green)
      &
      AF(carstate[2] = WAITING -> AF(carstate[2] = CLEARED & light.state=green) )
   )

CTLSPEC
   AG (
      AF(carstate[3] = CLEARED & light.state = green)
      &
      AF(carstate[3] = WAITING -> AF(carstate[3] = CLEARED & light.state=green) )
   )

CTLSPEC
   AG (
      AF(carstate[4] = CLEARED & light.state = green)
      &
      AF(carstate[4] = WAITING -> AF(carstate[4] = CLEARED & light.state=green) )
   )
```

References

1) https://pysathq.github.io/
2) https://www.geeksforgeeks.org/print-matrix-elements-from-top-left-to-bottom-right-in-diagonally-upward-manner/
3) https://pysathq.github.io/docs/html/api/card.html#pysat.card.ITotalizer
4) https://github.com/Z3Prover/z3
5) https://nusmv.fbk.eu/